Chapter 3 Solutions

CH03-Q01

`0x7f + 0x01` produces `0x80`, signed −128. Its mathematical value 128 exceeds 127, so signed overflow occurs.

`0x80 + 0xff` is mathematically −128−1=−129. Its low bits are `0x7f`, signed 127. Signed overflow occurs.

For `0x80 - 0x01`, compute `0x80 + 0xfe + 1`. The low result is again `0x7f`, with signed overflow. Using `0xff` as the complement of `0x01` would be an incorrect subtraction trace.

Signed overflow is $c_{\text{into MSB}} \oplus c_{\text{out of MSB}}$. Unsigned overflow uses only the outgoing carry.

CH03-Q02

Round	Old Q0	Add result C:A	A after shift	Q after shift
1	1	`0:1101`	`0110`	`1101`
2	1	`1:0011`	`1001`	`1110`
3	0	`0:1001`	`0100`	`1111`
4	1	`1:0001`	`1000`	`1111`

Round 2 shifts full input `1:0011:1101` to low eight bits `10011110`. The carry enters A; it is not accumulated a second time. The final concatenation is `(A,Q)=10001111`, representing $143 = 13 \times 11$.

Fix the initial multiplier $q < 2^n$, and define $R_k = 2^n A_k + Q_k$ after round k. The exact invariant is

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)2^{n-k}, \qquad 0 \le k \le n.$$

At $k = 0$, $A_0 = 0, Q_0 = q$, so it holds. When $k < n$, the second term is even. The current low bit is therefore original multiplier bit q_k. The conditional addition produces full combined value $R_k + q_k M 2^n$, including the carry. Shifting gives

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1})2^{n-k-1}.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 Solutions (continued)

The equality uses $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$, completing induction. At $k = n$, $q < 2^n$ makes the first term zero, so $R_n = Mq$. A contains the high half and Q the low half. Carry C enters A during the shift and is not lost. The processed old Q0 leaves; old Q1 becomes the next decision bit.

For this case R0..R4 are 11,109,158,79,143, matching the table.

$$T_{\text{serial}} = 3 + 32(1 + 1) + 2 = 69 \text{ cycles.}$$

An initiation interval of four cycles gives $1/4 = 0.25$ multiplication/cycle while a result may still take 69 cycles. Overlap requires separate partial-product/control state, collision handling, and contexts or stage resources for concurrent operations; merely duplicating the ALU is insufficient.

CH03-Q03

Round	Shifted A,Q	Trial A−M	State after decision
1	`00001,1010`	−4	Restore, `00001,1010`
2	`00011,0100`	−2	Restore, `00011,0100`
3	`00110,1000`	1	Keep and set bit, `00001,1001`
4	`00011,0010`	−2	Restore, `00011,0010`

Thus $Q = 2, R = 3$. Check $13 = 2 \times 5 + 3$ and $0 \le 3 < 5$.

The signed boundary is $-2147483648 / -1 = 2147483648$, above the signed maximum. In traditional MIPS32, `div` does not automatically provide an overflow trap with a dependable quotient. Software must check the operands. Other architectures may define a trap or another result.

CH03-Q04

$12.5 = 1100.1_2 = 1.1001_2 \times 2^3$. The sign is 1, stored exponent is $3 + 127 = 130 = 10000010_2$, and fraction is `10010000000000000000000`.

```
1 10000010 10010000000000000000000 = 0xc1480000
```

For `0x3f000000`, sign is 0, exponent is 126, and fraction is zero. Value is $1.0 \times 2^{-1} = 0.5$.

The smallest normal has exponent field `00000001` and zero fraction, hence 2^{-126}. The smallest subnormal has zero exponent field and only fraction bit 0 set, hence $2^{-126}2^{-23} = 2^{-149}$.

Chapter 3 Solutions (continued)

`0x7f800000` is positive infinity. `0x7fc00001` has all-ones exponent and a nonzero fraction with the quiet bit set, so it is a quiet NaN in this binary32 convention.

CH03-Q05

Binary32 spacing above 1 is 2^{-23}. The candidates are 1 and $1 + 2^{-23}$. The exact sum lies halfway. Candidate 1 has an even low retained significand bit, so nearest-even returns 1, encoding `0x3f800000`.

The addition is inexact. Its flag records that rounding occurred, not the numerical error 2^{-24}. Trap enablement is a separate control.

A wider intermediate can retain $1 + 2^{-24}$. Subtracting 1 before the final conversion then gives 2^{-24}, exactly representable in binary32. Rounding the initial addition to binary32 first instead gives zero after subtraction.

CH03-Q06

$1.5 = 1.1_2 \times 2^0$, while $2^{-126} = 1.0_2 \times 2^{-126}$. Their significand product is 1.1_2, and their unbiased exponent sum is −126. The product is already normalized, so no left shift is needed.

Stored exponents contain biases. Adding them includes two biases; the result needs one. General multiplication may also require a one-bit right normalization and exponent increment, followed by another adjustment if rounding carries. Special and subnormal cases require separate rules.

CH03-Q07

Spacing above 2^{24} is 2. Adding 1 reaches a midpoint between 2^{24} and $2^{24} + 2$. Nearest-even chooses 2^{24}. Therefore $(x + y) - x = 0$.

The other grouping computes $x - x = 0$, then adds 1, giving 1. The first rounding has lost the unit before cancellation can recover it.

Under the specified wider-intermediate assumption, $(1 + 2^{-24}) - 1 = 2^{-24}$. With binary32 rounding after each operation it is zero. This conclusion depends on retained precision and control settings, not merely on an x87 label.

Chapter 3 Solutions (continued)

CH03-Q08

Binary latency is $3 + 32 + 2 = 37$ cycles. Radix-4 latency is $6 + 16(1.5) = 30$ cycle-equivalents. The fractional per-iteration model is an average or abstract timing model, not an assertion that a synchronous stage ends halfway through an ordinary clock cycle.

By the separate acceptance-interval assumption, both throughputs are $1/8 = 0.125$ operations/cycle, or 125 operations per 1000 cycles after filling. No clock frequency is supplied, so operations per second cannot be calculated.

Latency matters for dependent chains and critical paths. Throughput matters when enough independent operations keep resources occupied. A single ordinary iterative unit cannot sustain an eight-cycle interval with a 37-cycle operation without additional overlap or replication.

For a divider, digit selection, normalization, correction, sign handling, zero-divisor checking, and remainder formation remain. Per-iteration timing can differ. Early termination can make costs input-dependent, in which case average time requires an operand distribution.

CH03-Q09

At 1.0, the retained significand is `1.00000000000000000000000`. Adding 2^{-24} leaves GRS=`100`. This is a tie between 1 and $1 + 2^{-23}$; retained LSB zero selects 1, bits `0x3f800000`, with inexact set.

$3 \cdot 2^{-25} = 2^{-24} + 2^{-25}$ gives GRS=`110`. It exceeds half a retained unit, so rounding increments the fraction LSB and yields $1 + 2^{-23}$.

2^{-151} is below half the least positive subnormal 2^{-149}, so it rounds to +0. 2^{-150} is exactly halfway between +0 and 2^{-149}, selecting even +0, bits `0x00000000`. Both are tiny and inexact and set underflow and inexact.

Maximum finite is `0x7f7fffff`. Adding it to itself exceeds the range; default RNE returns `0x7f800000`, +infinity, with overflow and inexact. `0*infinity` is invalid and defaults to quiet NaN with invalid set, rather than zero.

CH03-Q10

The encodings are `a=b=0x3f800001`, with exponent 127 and fraction LSB one, and `c=0xbf800002`.

$$ab = (1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46}.$$

Chapter 3 Solutions (continued)

At exponent zero the binary32 spacing is 2^{-23}. The residual 2^{-46} is far below half a ULP, so the separate multiply rounds to $1 + 2^{-22}$. Adding its exact negative gives +0, bits `0x00000000`. The addition cannot recover bits discarded by the multiply.

The fused exact intermediate is

$$(1 + 2^{-22} + 2^{-46}) - (1 + 2^{-22}) = 2^{-46}.$$

This is a normal binary32 value with biased exponent $127 - 46 = 81 = 0x51$, zero sign and fraction, hence `0x28800000`. A single rounding preserves a value lost by the separate product rounding.

At $x = 2^{24}$, ULP=2. `x+1` is a midpoint and rounds to even x, so `(x+1)-x=0`. Computing `x-x` first gives exact zero, then adding 1 gives 1. The different first rounding point causes the discrepancy. Each binary32 operation retains 24 significand bits; real associativity does not bypass these rounding operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 3 Solutions (continued)

Verification code appendix: ieee754_explorer.py

```
import math
import struct


def bits_to_f32(bits: int) -> float:
    return struct.unpack(">f", struct.pack(">I", bits))[0]


def f32_to_bits(value: float) -> int:
    try:
        return struct.unpack(">I", struct.pack(">f", value))[0]
    except OverflowError:
        return 0xFF800000 if value < 0 else 0x7F800000


def decode_f32(bits: int) -> dict[str, int | float | str]:
    sign = bits >> 31
    exponent = (bits >> 23) & 0xFF
    fraction = bits & 0x7FFFFF
    if exponent == 0xFF:
        kind = "infinity" if fraction == 0 else "NaN"
    elif exponent == 0:
        kind = "zero" if fraction == 0 else "subnormal"
    else:
        kind = "normal"
    return {
        "sign": sign,
        "exponent_field": exponent,
        "fraction_field": fraction,
        "kind": kind,
        "value": bits_to_f32(bits),
    }


def round_nearest_even(guard: int, round_bit: int, sticky: int, lsb: int) -> bool:
    """Return whether the retained significand increments."""
    return bool(guard and (round_bit or sticky or lsb))


def main() -> None:
    assert f32_to_bits(-12.5) == 0xC1480000
    assert decode_f32(0x3F000000)["value"] == 0.5
    assert decode_f32(0x00000001)["kind"] == "subnormal"
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 Solutions (continued)

```
    assert decode_f32(0x00000001)["value"] == math.ldexp(1.0, -149)
    assert decode_f32(0x00800000)["value"] == math.ldexp(1.0, -126)
    assert decode_f32(0x7F800000)["kind"] == "infinity"
    assert decode_f32(0x7FC00001)["kind"] == "NaN"

    rounded_once = bits_to_f32(f32_to_bits(1.0 + math.ldexp(1.0, -24)))
    assert rounded_once == 1.0

    x = float(2**24)
    left_grouping = (bits_to_f32(f32_to_bits(x + 1.0)) - x)
    right_grouping = (x - x) + 1.0
    assert left_grouping == 0.0
    assert right_grouping == 1.0
    assert left_grouping != right_grouping

    assert round_nearest_even(guard=1, round_bit=0, sticky=0, lsb=0) is False
    assert round_nearest_even(guard=1, round_bit=1, sticky=0, lsb=0) is True

    a = bits_to_f32(0x3F800001)  # 1 + 2^-23
    c = bits_to_f32(0xBF800002)  # -(1 + 2^-22)
    separate_product = bits_to_f32(f32_to_bits(a * a))
    assert f32_to_bits(separate_product) == 0x3F800002
    assert separate_product + c == 0.0
    assert f32_to_bits(math.ldexp(1.0, -46)) == 0x28800000

    assert f32_to_bits(math.ldexp(1.0, -150)) == 0x00000000
    assert f32_to_bits(bits_to_f32(0x7F7FFFFF) * 2.0) == 0x7F800000

    print(f"-12.5 bits: 0x{f32_to_bits(-12.5):08x}")
    print(f"smallest subnormal: {decode_f32(0x00000001)['value']:.17e}")
    print(f"1 + 2^-24 rounds to: {rounded_once}")
    print(f"grouping comparison: {left_grouping} != {right_grouping}")
    print(f"separate mul+add: 0x{f32_to_bits(separate_product + c):08x}")
    print(f"fused exact result: 0x{f32_to_bits(math.ldexp(1.0, -46)):08x}")
    print("IEEE-754 checks: PASS")


if __name__ == "__main__":
    main()
```

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study